

Построение пайплайнов данных

Основы архитектуры пайплайнов данных

Малахов Дмитрий Андреевич

Новосельцев Максим Сергеевич

23 января 2026 года

Из чего состоит курс

Блок 1: **Фундамент и анализ данных** (3 занятия)

Блок 2: **Алгоритмы и обработка данных** (6 занятий)

Блок 3: **DataOps и инфраструктура** (3 занятия)

Блок 4: **Надёжность, безопасность и качество** (2 занятия)

Итоговый проект (15-е занятие)

Роль архитектора данных в системе

Что делает архитектор данных?

Чем его работа выделяется на фоне других ИТ-специалистов?



Архитектор данных отвечает за то, как вся информация в компании или проекте живёт, перемещается и меняется

- Проектирование моделей данных
- Определение стандартов и правил работы с данными
- Выбор инструментов хранения и обработки данных
- Интеграция разнородных источников информации
- Контроль качества данных и безопасности
- Оптимизация производительности и масштабируемости

Ключевые навыки архитектора данных

- Знания о моделировании
 - ?
- Знания технологий хранения информации
 - ?
- Знания технологий обработки информации
 - ?
- Системное мышление
- Коммуникационные навыки



Связи с другими ролями

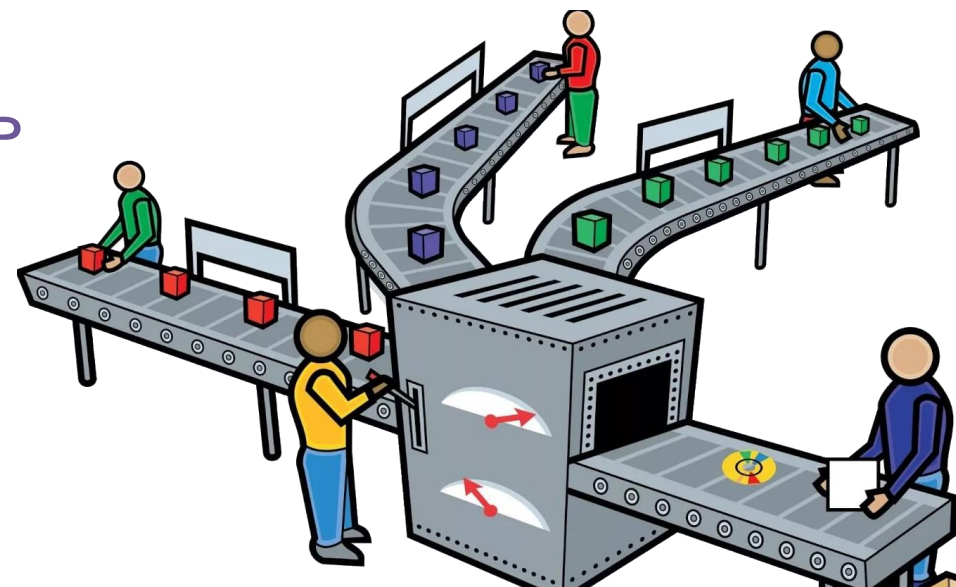
Архитектор данных не живёт в вакууме

- Архитектор информационных систем
- DevOps-инженер
- Дата-инженер
- Бизнес-аналитик
- Безопасник



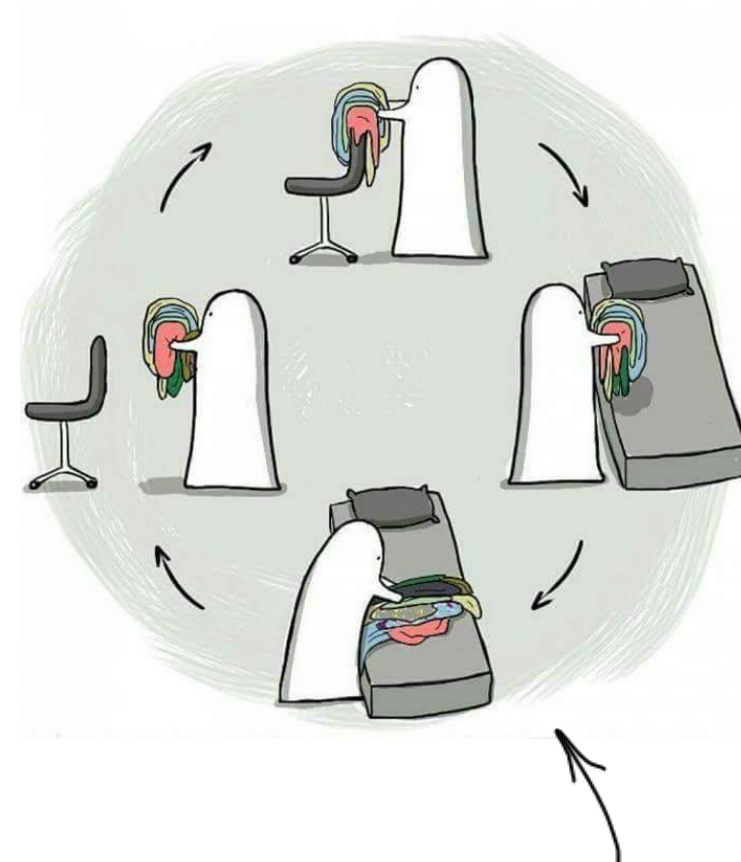
Результаты работы архитектора данных

- Единая документация
- Стратегия развития данных
- Внедрённые стандарты
- Автоматизация (но в меру)
- Измеримое качество и надёжность



Жизненный цикл анализа больших данных

1. Сбор данных
2. Предобработка данных
3. Анализ данных
4. Визуализация данных
5. Интерпретация результатов
6. Внедрение и поддержка решений



как не надо

Роль этапа сбора данных

Закладываем основу для всего последующего анализа

- Веб-сайты
- Транзакционные системы
- Социальные сети
- Сенсоры
- ...



Инструменты и технологии для хранения и обработки больших данных

- **Обработка данных**
 - Spark
- **Хранение**
 - Hadoop HDFS
 - Apache Ozone
 - S3
- **Форматы данных**
 - Apache Parquet
 - Apache Avro
 - Delta Lake, Apache Iceberg
- **Запросы и аналитика**
 - Trino
 - Apache Hive
 - ClickHouse
- **Потоковая передача данных**
 - Apache Kafka
 - Apache Pulsar
- **Оркестрация и управление**
 - Apache Airflow
 - Argo Workflows
 - Terraform
 - Ansible
- **Мониторинг и качество**
 - Grafana
 - Elasticsearch
 - Logstash
 - Kibana

И это далеко не все

Этап предобработки данных

- подготовить данные к анализу, чтобы они были корректными и не содержали искажений.

Этот этап особенно важен, так как качество данных напрямую влияет на точность и надежность результатов анализа

- Удаление дубликатов
- Исправление ошибок
- Фильтрацию шума
- Единый формат

Анализ данных и используемые методы

- Статистический анализ
- Машинное обучение
- Глубокое обучение



Визуализация данных и интерпретация результатов

- важный этап, на котором полученные данные превращаются в конкретные рекомендации и действия

- Требуется глубокого понимания бизнес-контекста и целей анализа
- Позволяет разработать стратегии, направленные на повышение эффективности и конкурентоспособности организации

Архитектурные уровни данных

Наиболее популярная и устоявшаяся в IT-мире модель выделяет три уровня:

- Физический уровень
- Логический уровень
- Внешний уровень

Связь между уровнями и независимость данных

- Логическая независимость данных
- Физическая независимость данных



Другие подходы и уровни

- Интеграционный слой
- Операционный слой
- Аналитический слой
- Визуализационный слой



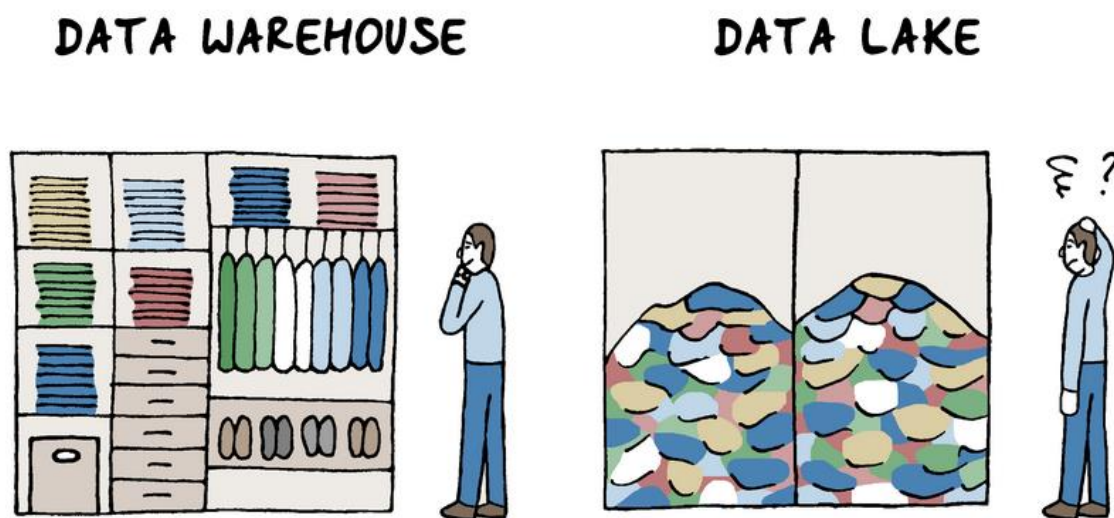
Data Warehouse

Типичная структура DWH

- Сырьевой слой
- Корпоративный слой
- Витрины данных

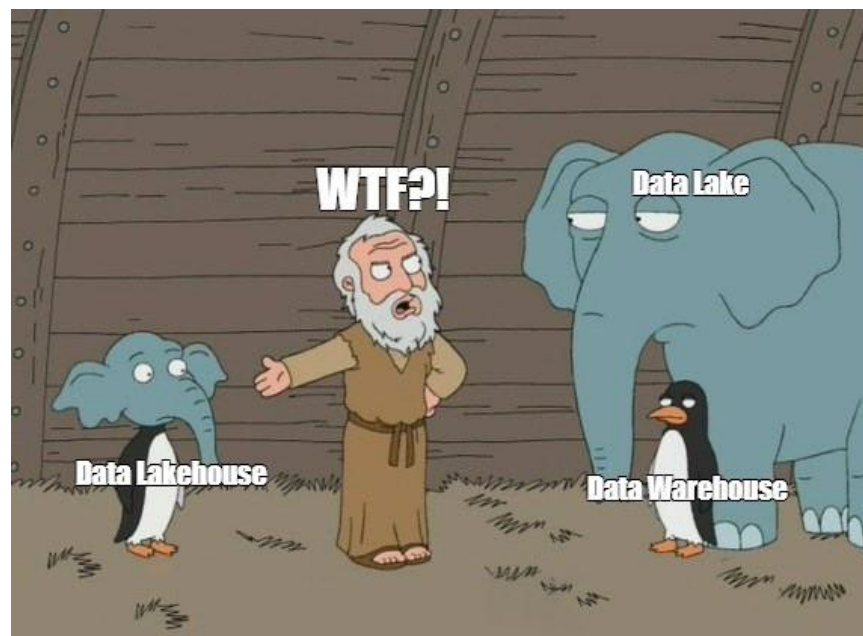
Data Lake

— это огромное, по сути бесконечное хранилище (обычно в облаке), куда попадают данные во всех видах и форматах, без требований к структуре.



Lakehouse

— это когда мы храним данные в том же «озере» (например, всё на Amazon S3 или аналогах), но при этом используем технологии, которые позволяют прямо на этих данных строить таблицы со схемами, строгими типами и поддержкой транзакций.



Принципы построения архитектуры данных

Нужна, чтобы:

- Данные были доступны нужным пользователям и приложениям
- Информация хранилась безопасно и не терялась
- Было понятно, где что лежит и как это ищется
- Систему можно было наращивать и менять без боли
- Дублирование и хаос не росли со скоростью света

Главные принципы построения архитектуры данных

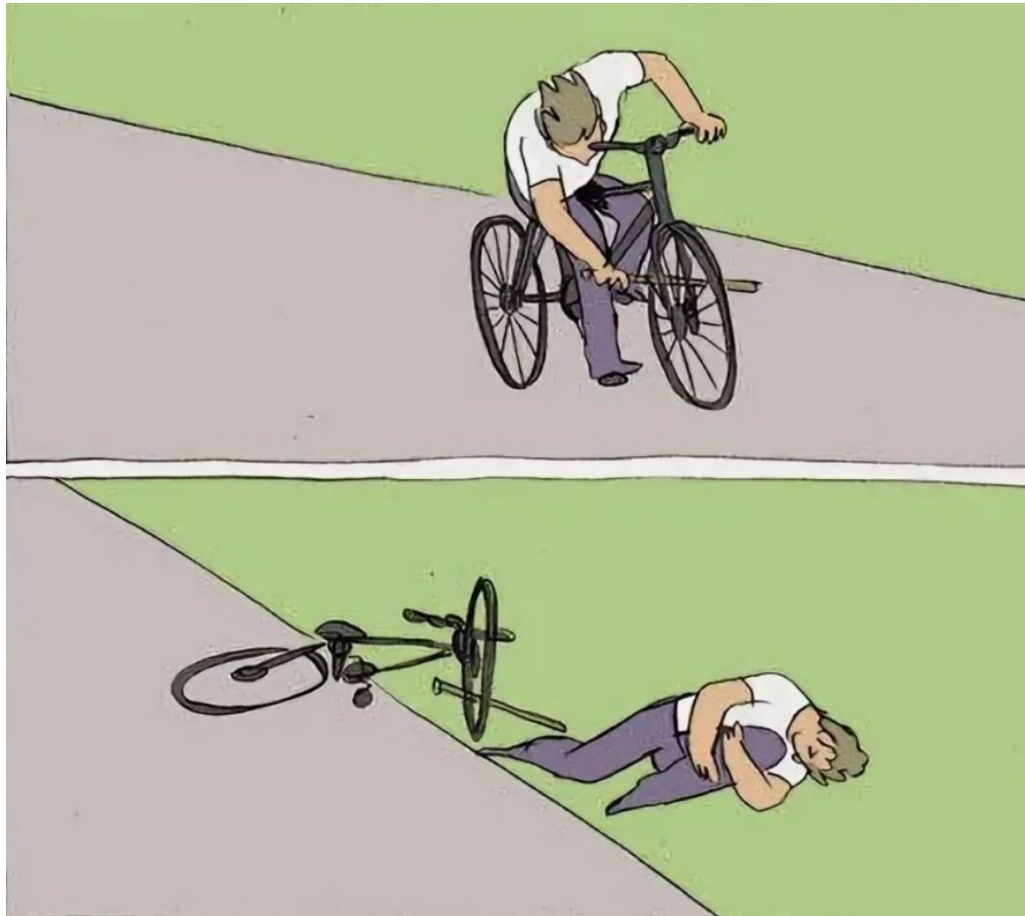
1. Принцип целостности данных (Data Integrity)
2. Принцип доступности (Availability)
3. Безопасность данных (Security)
4. Масштабируемость (Scalability)
5. Эффективность хранения (Storage Efficiency)
6. Гибкость и адаптивность (Flexibility & Adaptability)
7. Отвязка логической и физической архитектуры

Роли и артефакты в проектировании архитектуры данных

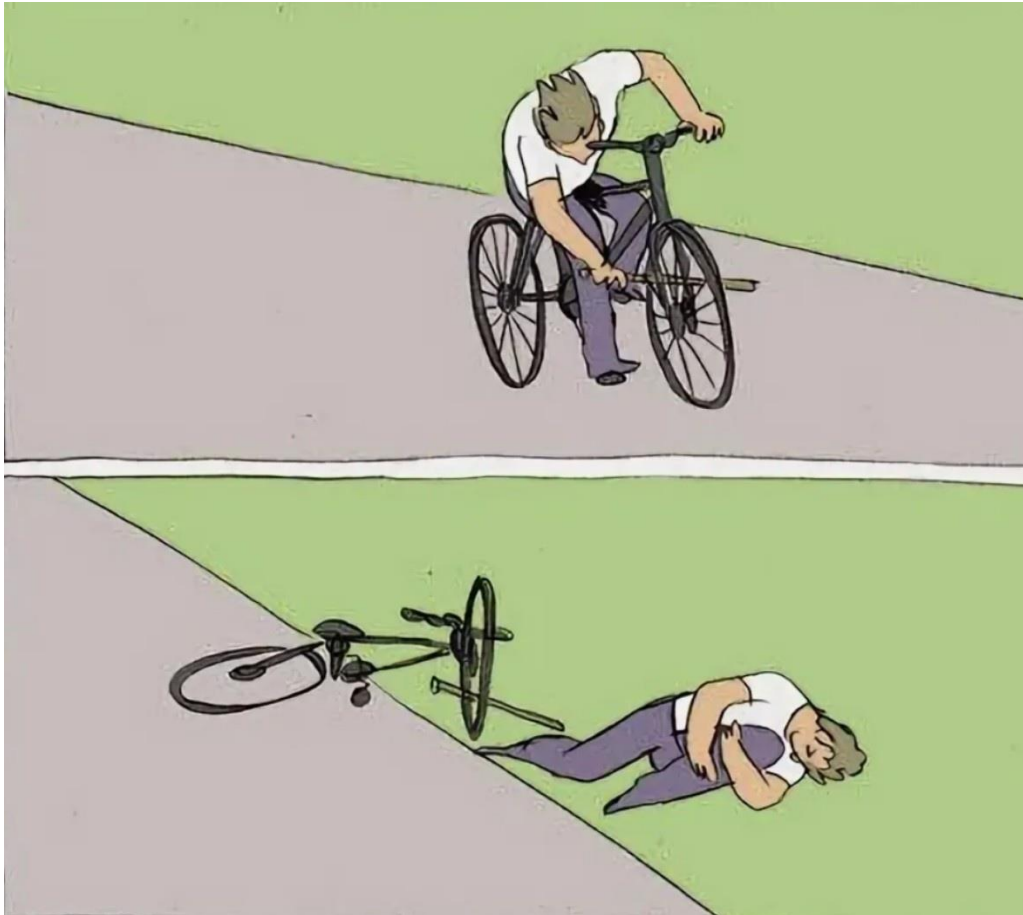
Три уровня архитекторов:

- Business
- Data
- Solution

Классические ошибки проектирования архитектуры данных



Классические ошибки проектирования архитектуры данных



- Избыточная сложность
- Избыточная централизация
- Отсутствие документации

Основные параметры качества данных

- Точность (accuracy)
- Полнота (completeness)
- Своевременность (timeliness)
- Доступность (availability)
- Согласованность (consistency)
- Однозначность (uniqueness)
- Актуальность (relevance)

Консистентность данных

Если в одном месте обновили, а в остальных нет —
начинается хаос.



Целостность данных

- Ссылочная целостность
- Уникальность
- Доменная целостность
- Семантическая целостность

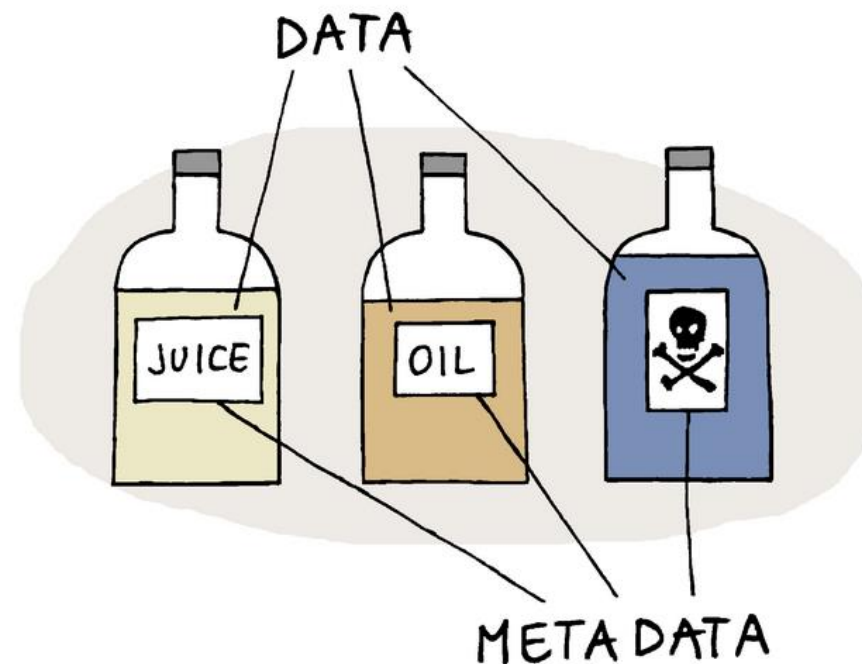
Как архитекторы обеспечивают качество данных

- Внедряют автоматические проверки при загрузке и обработке данных
- Определяют четкие правила на уровне схемы данных
- Описывают роли и ответственность
- Делают резервные копии и аудиты

Метаданные

Виды и уровни метаданных

- Технические
- Бизнес-метаданные
- Операционные
- Семантические
- Происхождение



Контракты данных

Типовые элементы контракта данных

- Структура данных
- Семантика
- Правила обновления
- Форматы передачи
- Версионирование
- Ошибки и валидация



Контракты данных

```
{  
  "type": "object",  
  "properties": { },  
}
```

Контракты данных

```
{  
  "type": "object",  
  "properties": {  
    "order_id": { "type": "string" },  
    "order_amount": { "type": "number", "minimum": 0 },  
    "order_date": { "type": "string", "format": "date" } },  
  "required": ["order_id", "order_amount", "order_date"]  
}
```


Потоки данных

— это путь, по которому данные проходят в информационной системе

1. Источник: база данных, веб-приложение, IoT-девайс
2. Транспортировка: шины, очереди, API
3. Обработка: ETL, скрипты, бизнес-логика
4. Хранилища или витрины
5. Потребители: BI-отчеты, ML-модели, внешние клиенты

Data Lineage

— это паспорт движения данных

- Аналитика и BI
- Управление качеством данных
- Отчетность и аудит



Проектирование пайплайна данных

Ответьте на вопросы по каждому блоку в вашем архитектурном решении:

1. Источники данных

- Какие именно источники? (Логи веб-серверов? События из мобильного приложения? Транзакции из БД? Внешние API?)
- Какой формат данных на выходе из источника?
- Какой предполагаемый объем? (Сколько ГБ/ТБ в день? Сколько событий/записей в секунду?)
- Какова критичность источника? (Можно ли потерять часть данных? Насколько важна свежесть?)

2. Сбор и интеграция данных

- Как данные будут поступать в систему?
- Как обеспечить надежность приема?
- Как обрабатывать разные форматы источников?

3. Хранилища и озёра данных

- Где и в каком формате будут храниться неизменённые исходные данные?
- Где будут лежать обработанные, структурированные данные?

4. Обработка и трансформация

- Какие основные трансформации нужны? (Очистка, обогащение, дедубликация, агрегация?)

5. Каталоги данных и метаданные

- Как потребители найдут нужные данные?
- Где будет храниться информация о схемах таблиц и их местоположении?
- Как отследить происхождение данных?

6. Качество и управление данными

- Какие проверки качества данных будут?
- Когда и как запускаются проверки?
- Что делать, если проверка не пройдена?

7. Безопасность и управление доступом

- Как аутентифицируются и авторизуются пользователи/системы?
- Как разграничить доступ к разным слоям данных?

5. Оркестрация и автоматизация

- Что будет запускать и координировать весь пайплайн?
- Как управлять расписанием и зависимостями задач?
- Как контролировать версию определенной задачи?